SOFTWARE REQUIREMENTS SPECIFICATION (SRS) v0.0.5

Project: AMC Movie Coordination Service (Discord Integration)

Document Version: v0.0.5

Status: Approved for Development

1. INTRODUCTION

1.1 Purpose

This document defines the architectural and functional specifications for a custom Discord bot integration. The service is designed to facilitate group coordination for cinematic events at AMC locations by aggregating local showtimes, synthesizing third-party review data and metadata, executing automated consensus-gathering (polling) mechanisms, communication thread, and event creation and notifications within a Discord server environment.

1.2 Scope

The application operates as an asynchronous, event-driven microservice interfacing directly with the Discord API. To circumvent the limitations of AMC's B2B API access and eliminate brittle DOM web scraping, the service employs a Data Ingestion module utilizing SerpApi (gl="us", hl="en") to extract theater schedules dynamically into a persistent weekly JSON cache (weekly_schedule.json). Secondary metadata lookups leverage zero-key public APIs (OMDb mirror and Wikipedia REST summary endpoints) to dynamically enrich entries with MPAA ratings, genres, and compact thumbnail posters. The system relies on local state caching for performance optimization (~4 API queries/month), features role-based access control (RBAC) for configuration, and utilizes parameter-driven payload visibility to manage user interface (UI) clutter.

1.3 Definitions and Acronyms

- **SerpApi:** Google Search Engine API provider used for localized showtime ingestion
- **RBAC:** Role-Based Access Control
- **TTL:** Time-to-Live (referencing cache invalidation)
- **OMDb:** Open Movie Database
- **REST:** Representational State Transfer
- **CLI:** Command Line Interface (mapped to Discord Slash Commands)

2. SYSTEM ARCHITECTURE

2.1 Technology Stack

- **Runtime Environment:** Python 3.10+

- **Asynchronous Framework:** asyncio / urllib (Non-blocking I/O operations and executor-wrapped network requests)
- **API Wrapper:** discord.py (WebSocket and REST API communication, slash commands, background tasks)
- **Data Ingestion Engine:** SerpApi (Google Search Engine API wrapper)
- **Metadata Fallbacks:** OMDb REST API Mirror / Wikimedia REST API (Page Summary endpoints)
- **State Management:** Ephemeral JSON-based document storage (Configuration and Cache)

2.2 High-Level Architecture

The system is divided into three decoupled components:

1. **Ingestion & Enrichment Service:** An independent module responsible for SerpApi network requests, day-resolution logic, and zero-key metadata extraction (genres, ratings, thumbnail poster URLs) via OMDb and Wikipedia REST endpoints.
2. **Discord Gateway (Event Loop):** The primary WebSocket connection handling command routing, autocomplete generation, visibility parameter enforcement, and embed payload construction.
3. **Task Scheduler:** A daemonized background process handling TTL-based cache invalidation, scheduled poll execution, and weekly Thursday 8:30 AM cache synchronizations.

3. FUNCTIONAL REQUIREMENTS (FRs)

3.1 Epic: Data Ingestion & Synchronization

- **FR-1.1:** The service shall execute a scheduled search query (q="{theater_name} showtimes") via SerpApi using parameters gl="us" and hl="en" to extract daily operational schedules for configured theater locations.
- **FR-1.2:** The service shall serialize and commit extracted showtime heuristics to a local JSON cache (weekly_schedule.json) to minimize external network calls, prevent rate-limiting, and limit execution costs to ~4 API calls/month.
- **FR-1.3:** During schedule ingestion, the service shall query OMDb mirror and Wikipedia REST summary endpoints to resolve missing genres, MPAA ratings, and direct .jpg/.png thumbnail poster URLs.
- **FR-1.4:** In the event of unresolved external metadata, the system shall assign explicit fallback placeholders: "Unknown" for missing genres or ratings, and a default AMC placeholder image URL for missing posters, preventing misleading default labels.
- **FR-1.5:** The service shall implement fault tolerance for network exceptions. Upon API error or parsing failure, the system shall construct and dispatch an alert payload via Direct Message (DM) to the designated Server Administrator.

3.2 Epic: Configuration & State Management (Admin)

- **FR-2.1:** The service shall expose a /set_theater command restricted to users possessing server administration permissions (RBAC validation).

- **FR-2.2:** The service shall leverage the Discord API autocomplete callback, querying local configuration state to provide sub-second typeahead suggestions during theater selection.
- **FR-2.3:** The service shall persist the selected location's unique name string and optional geographical location to a local configuration state file.
- **FR-2.4:** A scheduled asynchronous task shall execute weekly to invalidate and refresh the schedule cache, ensuring delta synchronization for newly operational or updated movie showtimes.
- **FR-2.5:** The service shall expose a /force_refresh command restricted to administrators to manually trigger an immediate re-ingestion of showtimes and metadata.

3.3 Epic: Client Interfaces & Query Resolution

Note: All client-invoked commands must default to an ephemeral state (private UI payload) to optimize chat cleanliness, utilizing an optional overriding string parameter for state transition.

- **FR-3.1:** The service shall expose a /showtimes command accepting a required week_day choice selection that retrieves and embeds the operational schedule from the local cache. The payload shall display a dynamically calculated DD/MM/YYYY target date computed relative to the weekly cache update timestamp. It must utilize a limitless pagination architecture that dynamically chunks showtimes across embed fields and multiple messages to safely bypass Discord API character, field, and embed constraints.
- **FR-3.2:** The service shall expose a /movies command accepting one primary argument: movie_title (leveraging typeahead autocomplete bound to the current weekly schedule cache) to look up a specific film and automatically filter out showtimes that have already passed in real-time.
- **FR-3.3:** The service shall expose a /upcoming command utilizing updated API ingestion to parse and return future theatrical releases mapped to the active configuration state.
- **FR-3.4:** The service shall expose a /review command that accepts a movie parameter (leveraging typeahead autocomplete bounded to the current weekly schedule cache) that queries the external OMDb REST API, parses the JSON response, and constructs a Discord Embed containing synthetic aggregate scores (Rotten Tomatoes, Metacritic).
- **FR-3.5:** The service shall implement a !help command to provide an interactive guild manual detailing available commands, cache schedules, and usage flags.
- **FR-3.6:** All client commands shall accept an optional string parameter visible_to_all.
- **FR-3.7:** The gateway shall evaluate the visibility parameter. If the string evaluates strictly to “true”, the gateway shall set ephemeral=False, broadcasting the payload to the channel. In all other cases (including null/None), the gateway shall default to ephemeral=True.

3.4 Epic: Automated Event Scheduling

- **FR-4.1:** The service shall initialize a background daemon utilizing discord.ext.tasks.
- **FR-4.2:** On a configurable, recurring cron schedule (weekly on Thursdays at 8:30 AM), the daemon shall invoke the ingestion module targeting a forward-looking date parameter (upcoming week).
- **FR-4.3:** The service shall parse the aggregate showtime data, select a subset of highly-indexed operational titles, and construct a native Discord UI Poll object.

- **FR-4.4:** The service shall dispatch the Poll object to a pre-configured, persistent channel ID designated for group consensus.
- **FR-4.5:** Upon the conclusion of a successful movie and time slot vote, the system shall automatically generate a scheduled Discord Event for the winning selection.
- **FR-4.6:** The system shall automatically create a dedicated Public Thread linked to the event or announcement to isolate spoiler-friendly discussion and coordination.
- **FR-4.7:** Whenever a poll, discussion thread, or event is generated, the system shall dynamically mention a designated 'movie goer' role to alert opted-in users without pinging the entire server.

4. NON-FUNCTIONAL REQUIREMENTS (NFRs)

4.1 Performance & Concurrency

- **NFR-1 (Asynchronous I/O):** All external HTTP requests, SerpApi queries, and file I/O operations must be awaited or executed inside asynchronous thread executors (e.g., asyncio.to_thread()) to prevent blocking the primary event loop, ensuring concurrent processing of multitenant commands.
- **NFR-2 (Latency):** Autocomplete queries and schedule retrievals querying the local JSON cache must resolve and return payloads to the Discord API within 500ms to ensure a seamless UI/UX.

4.2 Fault Tolerance & Graceful Degradation

- **NFR-3 (Error Handling):** In the event of an external API timeout, 4xx/5xx status code, or parsing failure, the bot shall catch the exception and return a standardized, user-facing error embed rather than terminating the runtime process.
- **NFR-4 (Dependency Resilience):** The discord gateway and internal event loops must remain fully operational even if external APIs or search engines are temporarily unreachable.

4.3 Security & Secrets Management

- **NFR-5 (Environment Variables):** Hardcoding of API keys, Discord tokens, or infrastructure secrets is strictly prohibited. All environment variables must be injected at runtime via an external .env file or CI/CD secrets manager.
- **NFR-6 (Least Privilege):** The application shall request only the necessary Discord OAuth2 scopes (e.g., application.commands, send_messages) and abstain from requesting elevated intents (like message_content) unless strictly required by the architecture.